新特性

- ## var的使用
  - 需要注意的是，var 只能用于局部变量，并且必须在声明的同时进行初始化，这样编译器才能推断出变量的类型。var 不能用于类成员变量、方法参数或返回类型。

```
public class TestDemo {

    public static void main(String[] args) {
        var i = "hello";
        System.out.println(i);
    }
}
```

- ## 文本块

```
/**
 * 使用JDK17返回HTML文本
 * @return 返回HTML文本
 */
public static final String getHtmlJDK17() {
    return """
        <html>
        <body>
        <p>Hello, world</p>
        </body>
        </html>
        """;
}
```

- ## Records

```
/**
 *
 * @param stuId     学生ID
 * @param stuName   学生名称
 * @param stuAge    学生年龄
 * @param stuGender 学生性别
 * @param stuEmail  学生邮箱
 */
public record StudentRecord(Long stuId,
                            String stuName,
                            int stuAge,
                            String stuGender,
```

```
                                String stuEmail) {
    public StudentRecord {
        System.out.println("构造函数");
    }

    public static void main(String[] args) {
        StudentRecord record = new StudentRecord(1L, "张三", 16, "男",
"xxx@qq.com");
        System.out.println(record);
    }
}
```

- **全新的switch表达式**

```
public int getByJDK17(Week week) {
        // 1, 现在的switch变成了表达式，可以返回值了，而且支持yield和->符号
来返回值
        // 2, 再也不用担心漏写了break，而导致出问题了
        // 3, case后面支持写多个条件
        return switch (week) {
            case null -> -1;
            case MONDAY -> 1;
            case TUESDAY -> 2;
            case WEDNESDAY -> 3;
            case THURSDAY -> {yield 4;}
            case FRIDAY -> 5;
            case SATURDAY, SUNDAY -> 6;
            default -> 0;
        };
    }

    private enum Week {
        MONDAY,
        TUESDAY,
        WEDNESDAY,
        THURSDAY,
        FRIDAY,
        SATURDAY,
        SUNDAY
    }
}
```

- **私有接口方法**

```
public interface PrivateInterfaceMethod {
    /**
     * 接口默认方法
     */
    default void defaultMethod() {
        privateMethod();
    }

    // 接口私有方法，在Java8里面是不被允许的
    private void privateMethod() {
    }
}
```

- **模式匹配**

```
/**
 * 转换并申请了一个新的变量，极大地方便了代码的编写
 *
 * @param value
 */
public void matchByJDK17(Object value) {
    if (value instanceof String v) {
        System.out.println("遇到一个String类型" + v.toUpperCase());
    } else if (value instanceof Integer v) {
        System.out.println("遇到一个整型类型" + v.longValue());
    }
}
```

- **集合类的工厂方法**

```
Set<String> set = Set.of("a", "b", "c");

List<Object> list = List.of(1, "2", "3");

Map<String, String> map = Map.of("key1", "value1", "key2", "value2");

Map<String, String> entries = Map.ofEntries(Map.entry("key1",
"value1"),

 Map.entry("key1", "value1"));
```

- **Stream增强**

```
// takeWhile 顺序返回符合条件的值，直到条件不符合时即终止继续判断，
// 此外toList方法的加入，也大大减少了节省了代码量，免去了调用
collect(Collectors::toList)方法了
List<Integer> list = Stream.of(2,2,3,4,5,6,7,8,9,10)
        .takeWhile(i->(i%2==0)).toList(); // 返回2, 2
```

```
// dropWhile 顺序去掉符合条件的值，直到条件不符合时即终止继续判断
List<Integer> list1 = Stream.of(2,2,3,4,5,6,7,8,9,10)
        .dropWhile(i->(i%2==0)).toList(); //返回3, 4, 5, 6, 7, 8, 9, 10

// ofNullable，支持传入空流，若没有这个且传入一个空流，那么将会抛NPE
var nullStreamCount = Stream.ofNullable(null).count(); //返回0

// 以下两行都将输出0到9
Stream.iterate(0, n -> n < 10, n -> n + 1).forEach(x ->
System.out.println(x));
Stream.iterate(0, n -> n + 1).limit(10).forEach(x ->
System.out.println(x));
```

- **string字符串新方法**
  - repeat：重复生成字符串
  - isBlank：不用在引入第三方库就可以实现字符串判空了
  - strip：去除字符串两边的空格，支持全角和半角，之前的trim只支持半角
  - lines：能根据一段字符串中的终止符提取出行为单位的流
  - indent：给字符串做缩进，接受一个int型的输入
  - transform：接受一个转换函数，实现字符串的转换

```
public class Demo {
    @Test
    public void test(){
        String str = "  hello world  ";
        // 重复字符串5次
        String repeat = str.repeat(5);
        System.out.println(repeat);
    }
    @Test
    public void test1(){
        //isBlank 更严格，它会检查空白字符，而 isEmpty 只会检查是否没有字符。
        String str = "    ";
        String str1 = "   ";
        // 是否为空
        System.out.println(str.isBlank());
        System.out.println(str1.isEmpty());
    }
    @Test
    public void test2(){
        //去掉全半角空格
        String str = "  hello  ";
        System.out.println(str.strip().length());
    }
    @Test
    public void test3(){
        //转为流
```

```
        String str = "  hello  ";
        Stream<String> lines = str.lines();
        lines.forEach(System.out::println);
    }
    @Test
    public void test4(){
        //缩进
        String str = "  hello  ";
        String indent = str.indent(10);
        System.out.println(indent);
    }
    @Test
    public void test5(){
        //接收一个转换函数，将字符串转换为其他类型
        String str = "  hello  ";
        String transform = str.transform(String::toUpperCase);
        System.out.println(transform);
    }
}
```